employee-data-project

May 19, 2025

1 Employee data for the Company

Employee data is an essential component of any organization's human resources and operational framework. It encompasses structured information about individuals working within the company, including personal details, job roles, compensation, departmental assignments, and performance metrics. Maintaining and analyzing this data allows companies to make informed decisions related to hiring, workforce planning, productivity tracking, and strategic growth.

Using employee data in a Python project provides a realistic, impactful, and multi-dimensional problem space that enables valuable learning and practical application of data science techniques.

We will ensure to solve several problem statements related to employee data that can be used for analytics, system development, process improvement, or research projects:

1) Performance Analysis
2) Gender vise salary Distribution.
3) Recruitment Funnel Optimization.
4) Diversity and Inclusion Assessment.

I have prepared a CSV file containing comprehensive employee data, which will assist in evaluating each individual's performance and role.

[38]:
```
# import pandas
import pandas as pd

# Define file contaning the dataset
Employee_data = 'datasets/Employee_data.csv'

#Created a Dataframe for the respected file
df_employee = pd.read_csv('Employee_data.csv')

# Selecting sample data for 3 random rows
display(df_employee.sample(3))

# print Dataframe summary
df_employee.info()
```

```
    EmployeeID          Name  Gender   Department  Salary JoiningDate  \
45         146   Layla Price     NaN    Marketing  117716   7/13/2020
73         174  Bella Torres     NaN  Engineering  140365   1/19/2018
19         120  Brian Walker     NaN    Marketing  180663    4/1/2022

    FullTime Shift timings  Age       Location  PerformanceScore
45     False  9 AM to 5 PM   30        Chicago              2.16
73      True  9 AM to 5 PM   33        Chicago              6.87
19     False  9 AM to 5 PM   38  San Francisco              8.53
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   EmployeeID        100 non-null    int64
 1   Name              100 non-null    object
 2   Gender            0 non-null      float64
 3   Department        100 non-null    object
 4   Salary            100 non-null    int64
 5   JoiningDate       100 non-null    object
 6   FullTime          100 non-null    bool
 7   Shift timings     71 non-null     object
 8   Age               100 non-null    int64
 9   Location          100 non-null    object
 10  PerformanceScore  100 non-null    float64
dtypes: bool(1), float64(2), int64(3), object(5)
memory usage: 8.0+ KB
```

[24]:
```
# import filter for warnings in python
import warnings
warnings.filterwarnings('ignore')
```

[25]:
```
#Using head method to show top 10 data from the table
df_employee.head(10)
```

[25]:
```
   EmployeeID           Name  Gender   Department  Salary JoiningDate  \
0         101       John Doe     NaN        Sales  167392   6/23/2019
1         102     Jane Smith     NaN  Engineering  132658   8/15/2020
2         103  Emily Johnson     NaN    Marketing  189470   1/10/2021
3         104  Michael Brown     NaN  Engineering  146215   11/1/2018
4         105    Linda Davis     NaN           HR  136793   3/30/2019
5         106   Chris Wilson     NaN        Sales  101469   7/19/2020
6         107    Nancy Allen     NaN  Engineering  119857   9/25/2021
7         108   Robert White     NaN    Marketing  172438  12/17/2019
8         109    Laura Green     NaN           HR  146599    4/8/2018
9         110    Kevin Scott     NaN        Sales  139527    1/5/2022
```

	FullTime	Shift timings	Age	Location	PerformanceScore
0	True	9 AM to 5 PM	23	New York	4.65
1	True	9 AM to 5 PM	28	San Francisco	3.56
2	False	9 AM to 5 PM	35	Chicago	4.11
3	True	NaN	30	Boston	5.27
4	False	9 AM to 5 PM	26	New York	5.06
5	True	9 AM to 5 PM	22	Chicago	2.74
6	True	9 AM to 5 PM	33	Boston	1.00
7	False	9 AM to 5 PM	40	Los Angeles	3.35
8	True	9 AM to 5 PM	24	New York	8.11
9	False	9 AM to 5 PM	27	San Francisco	9.82

2 2 . Data Preprocessing

Data preprocessing for employee data is a crucial step before any analysis, modeling, or reporting. It ensures the data is clean, consistent, and suitable for drawing insights.

During the data preprocessing stage, the use of the info() method revealed that several rows contained missing values, likely due to some employees not providing complete information. To address this, we filled in the missing data where appropriate and removed certain columns—such as Shift Timings and Age—that were deemed irrelevant for the current analysis.

To ensure accurate analysis and avoid potential bias, we will address the missing values in the Department and Salary columns at a later stage. For now, our initial data preprocessing steps will focus on:

1)Removing columns that are not relevant to our analysis.
3)Identifying and counting missing values across the dataset.

[26]:
```
# Define list of column to be deleted
cols_to_drop = ['Age','Location']
```

[27]:
```
# Delete unnecessary columns
df_employee.drop(cols_to_drop, axis=1, inplace=True)
```

[28]:
```
# Count missing values for each column
df_employee.isnull().sum().sort_values(ascending = False)
```

[28]:
```
Gender             100
Shift timings       29
EmployeeID           0
Department           0
Name                 0
Salary               0
JoiningDate          0
FullTime             0
```

```
PerformanceScore       0
dtype: int64
```

3 3. Dealing with missing values

We observed that there are missing values in the dataset, specifically in the following columns: a) Shift Timing and b) Gender.

For columns with repeating values, such as these, missing data can be imputed using the mode. Additionally, I experimented with a randomization approach to impute missing values in the 'Gender' column.

[30]:
```
# Removed the NAN value by using the Mode method
df_employee['Shift timings'] = df_employee['Shift timings'].
 ↪fillna(df_employee['Shift timings'].mode()[0])
```

[31]:
```
# Imported random library
import random

# Made a function named fill_gender_by_name to use it to assign the gender␣
 ↪accordingly
def fill_gender_by_name(df, name_column, gender_column):
    """Fills missing gender values based on name using a Database value."""
    # Loop through each row in the DataFrame
    for index, row in df.iterrows():
        # Check if the gender column value is missing (NaN)
        if pd.isnull(row[gender_column]):
            # Extract the name from the specified name column
            name = row[name_column]
            # Check if the name exists in the 'Name' dictionary
            if name in Name:
                # If yes, assign the corresponding gender from the dictionary
                df.loc[index, gender_column] = Name[name]
            else:
                # If the name is not found, assign gender randomly as either␣
 ↪'Male' or 'Female'
              df.loc[index, gender_column] = random.choice(['Male', 'Female'])
              #Assign randomly if name not in data
    return df

# Example usage:
Name = df_employee['Name']
df = df_employee
#We will use the above function to fill the gender coulmn
df_employee = fill_gender_by_name(df_employee, 'Name', 'Gender')
df_employee.isnull().sum().sort_values(ascending = False)
```

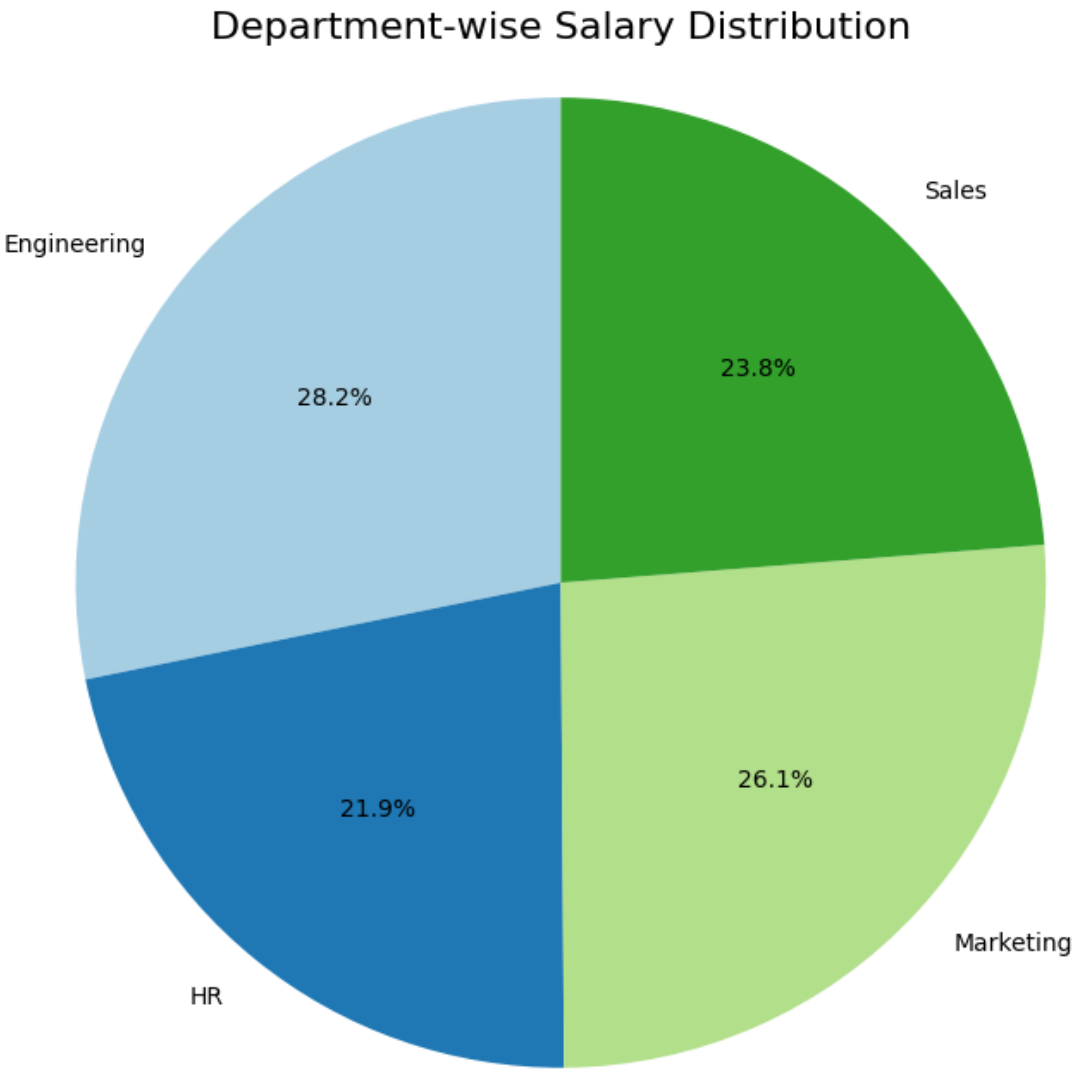
[31]:
```
EmployeeID          0
Name                0
Gender              0
Department          0
Salary              0
JoiningDate         0
FullTime            0
Shift timings       0
PerformanceScore    0
dtype: int64
```

4 4. Plot running data

By creating visualizations, we can effectively track how employee-related data evolves over time. These plots help identify workforce trends such as hiring patterns, salary distribution, and demographic shifts, including gender diversity.

4.1 Department-wise Salary Distribution

[32]:
```
#import matplotlib
import matplotlib.pyplot as plt

#import seaborn library
import seaborn as sns

# Group by department and sum salaries
department_salary = df_employee.groupby('Department')['Salary'].sum()

## Pie Chart
plt.figure(figsize=(8, 8))  # Adjust figure size for better visualization
plt.pie(department_salary, labels=department_salary.index, autopct='%1.1f%%',␣
 ↪startangle=90, colors=plt.cm.Paired.colors)

# Add a title
plt.title('Department-wise Salary Distribution', fontsize=16)

# Ensure a circular shape
plt.axis('equal')

# Display the chart
plt.show()
```

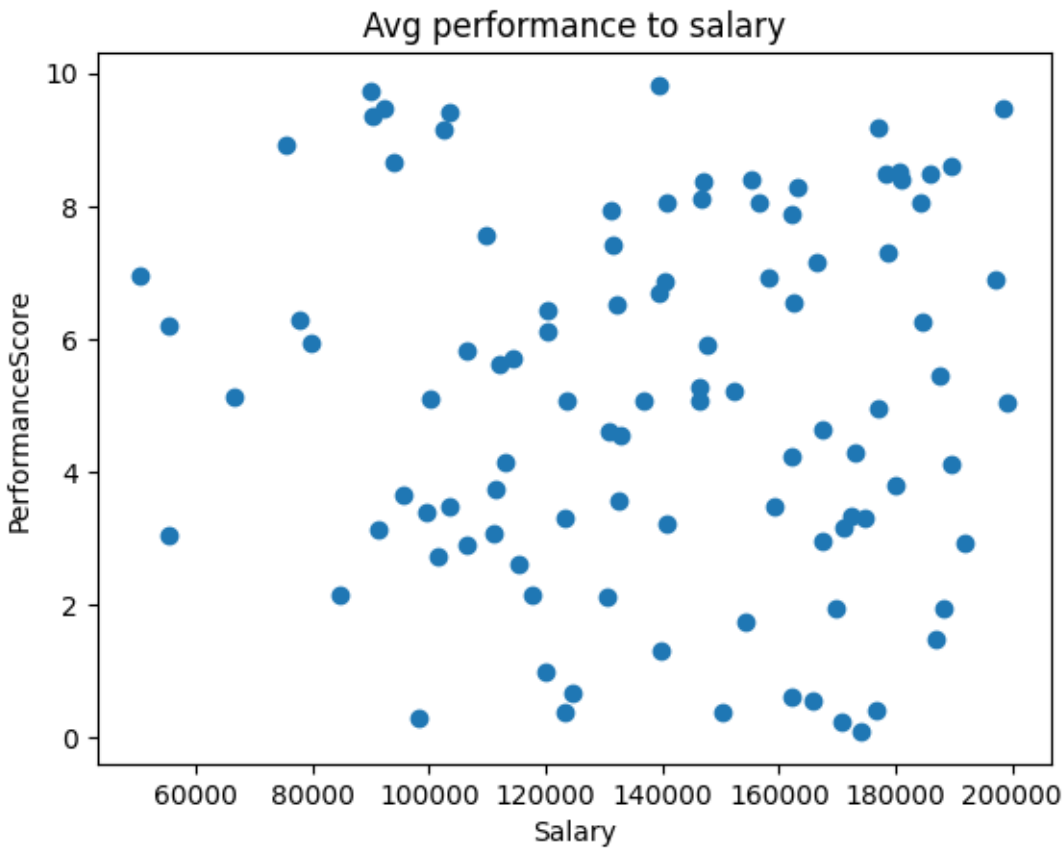


4.2 Average Salary by Gender

[33]:
```
#Bar Plot
sns.barplot(df_employee, x='Gender', y='Salary', estimator='mean')
plt.title('Average Salary by Gender')
plt.ylabel('Average Salary')
plt.show()
```

4.3 Avg performance to salary

[34]:
```
#Scatter Plot
plt.scatter(df_employee['Salary'],df_employee['PerformanceScore'])
plt.title('Avg performance to salary')
plt.xlabel("Salary")
plt.ylabel("PerformanceScore")
plt.show()
```



5 5. Employee Salary Statistics

Employee Salary Statistics provide a quantitative overview of salary distribution within the organization, using key metrics such as average, median, minimum, and maximum salaries to identify compensation trends and support data-driven decision-making.

[35]:
```
#First we will calculate the Average of the employee salary
Avg_salary = df_employee['Salary'].mean()
print('Average salary :', Avg_salary)

#Second the Median for the employee salary
ME_salary = df_employee['Salary'].median()
print('Median salary :',ME_salary)

#third the minimum salary for the employee data
min_salary = df_employee['Salary'].min()
print('Minimum salary :',min_salary)

#last one will be the maximum salary for the employee data
```

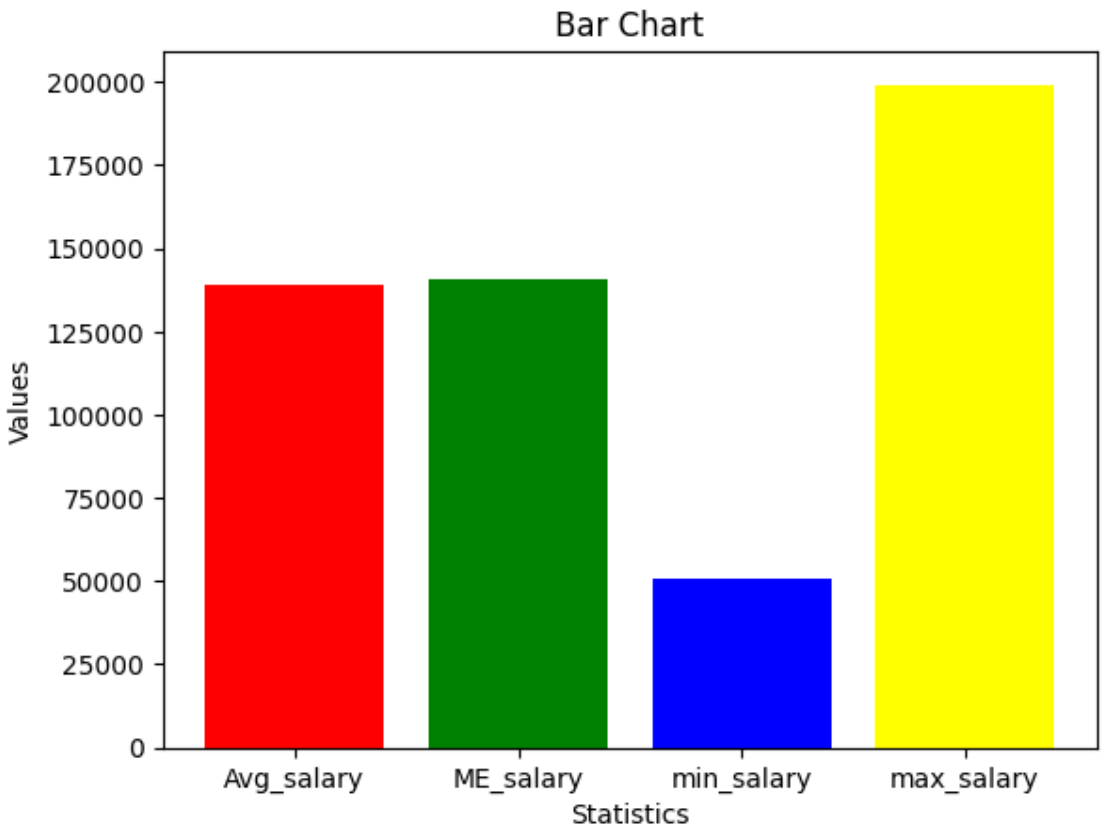
```
max_salary = df_employee['Salary'].max()
print('Maximum salary :',max_salary)
```

```
Average salary : 138969.61
Median salary : 140506.5
Minimum salary : 50511
Maximum salary : 199241
```

6 6. Data Visualization for Employee Salary Statistics

Data Visualization for Employee Salary Statistics involves the use of charts and graphs to present salary data clearly and effectively. This helps in identifying patterns, disparities, and trends across departments, roles, and locations, enabling more informed and strategic compensation decisions.

[36]:
```
#Bar graph to reprsent the Employee Salary Statistics
Statistics = ['Avg_salary','ME_salary','min_salary','max_salary']
Values = [138969.61, 140506.5,50511,199241]
plt.bar(Statistics, Values, color=['red', 'green', 'blue', 'yellow'])
plt.title("Bar Chart")
plt.xlabel("Statistics")
plt.ylabel("Values")
plt.show()
```

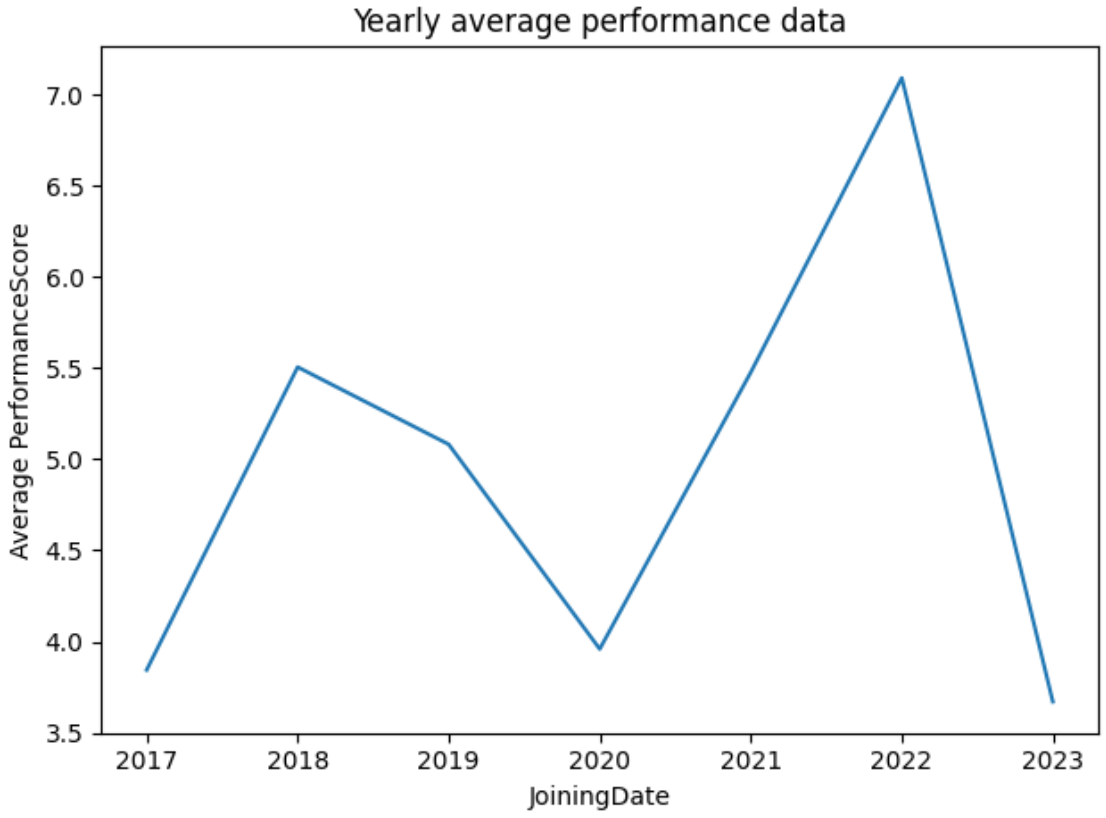


7 7. In-Depth Analysis of Employee Dataset

To uncover meaningful insights, trends, and patterns within employee data to inform decision-making around performance management, compensation, workforce planning, and organizational development.

We can conduct an analysis of the average annual performance of employees using the plot graph for easy understanding of the data.

[37]:
```
#converting my object data to datetype data
df_employee['JoiningDate'] = pd.to_datetime(df_employee['JoiningDate'])

#Representing the yearly average performance using line plot grapgh
df_employee.groupby(df_employee['JoiningDate'].dt.year)["PerformanceScore"].
 ↪mean().plot(kind="line")

# Adding labels
plt.xlabel("JoiningDate")
plt.ylabel("Average PerformanceScore")
plt.title("Yearly average performance data")
plt.tight_layout()
```



8 8. Key Observations and Strategic Insights

8.0.1 a) Overall Performance Trend

The average performance scores show fluctuating trends from 2017 to 2023, with both notable highs and lows.

8.0.2 b) Peaks in Performance

2022 recorded the highest average performance score of the entire period.

8.0.3 c) Decline Points

2020 saw a significant dip in performance scores.This could be associated with the global impact of the COVID-19 pandemic, remote work adaptation, or organizational disruption. A sharp drop in 2023 might indicate recent organizational or cultural challenges, onboarding newer employees with lower initial scores, or performance evaluation changes.

8.0.4 d) Initial Improvement

From 2017 to 2018, there was a strong upward trend, indicating early success in performance metrics or hiring high-performing individuals.

9 9. Conclusion

This project provides a comprehensive examination of employee data aimed at uncovering actionable insights to support HR strategies and business decisions. By leveraging structured datasets, we explored several core areas including performance trends, gender-based salary distributions, recruitment funnel efficiency, and diversity metrics.

9.0.1 Key findings include:

a) Fluctuations in yearly performance scores, with notable peaks in recent years that may reflect evolving HR practices or external factors.

b) Data preprocessing revealed gaps in information such as missing demographic or job-related entries, underlining the need for more robust data collection.

c) Salary distribution and performance metrics varied significantly across departments, locations, and employment types (full-time vs part-time), emphasizing the importance of contextual analysis in workforce planning.

d) Recruitment and inclusion metrics were partially addressed, but further enrichment of the dataset (e.g., complete gender and role data) is needed for more accurate diversity assessments.

Overall, this project not only demonstrates practical data analysis skills using Python and pandas, but also highlights the critical role of data-driven insights in shaping employee-related policies and improving organizational performance. Future work could include predictive modeling for turnover, deep-dive analysis of department-specific KPIs, or integration with external datasets to enhance insights.

[ ]: